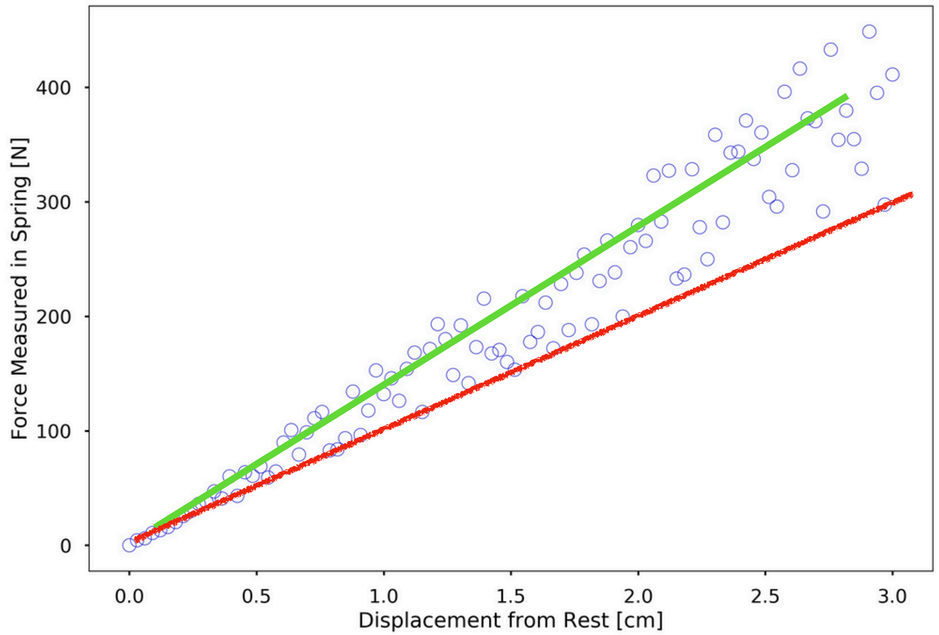
Least Squares Regression

- **What is Least Squares Regression?**
  - A mathematical method used to find the best-fitting line (or model) through a set of data points by minimizing the sum of the squares of the differences between the observed data and the predicted values.
  - Commonly used in scientific and engineering problems to estimate unknown parameters from experimental data.
- **Motivation**
  - In practical situations, model parameters, like a spring stiffness in a set of spring experiments, are not directly known. Instead, we rely on experimental data that usually contains noise.
  - Least Squares Regression helps in determining the most likely model parameters, even when the data contains errors or noise.
- **Example**: Determining spring stiffness k from experimental data of force F and displacement x in a linear spring system: $F = kx$.



Methods of Deriving Least Squares Regression

- **Two Key Derivations:**

  - **Linear Algebra Approach**: Solves the regression problem by constructing a system of linear equations based on the data and minimizing the residual sum of squares.

  - **Multivariable Calculus Approach**: Minimizes the residual sum of squares by taking derivatives and setting them to zero to find the optimal parameters.

- **Linear Model Example**: For a linear relationship $y = mx + b$, the goal is to minimize the error E, where

$$E = \sum (y_{\text{observed}} - y_{\text{predicted}})^2$$

  - The solution yields the optimal slope m and intercept b.

Least Squares Regression in Python and Nonlinear Extensions

- **Implementing in Python**
  - Python libraries like **NumPy** and **SciPy** provide efficient functions to perform least squares regression.
  - Example: Using numpy.linalg.lstsq() to solve a linear system or scipy.optimize.curve_fit() for nonlinear models.

- **Nonlinear Regression**
  - Least Squares can also be extended to nonlinear functions, such as fitting data to exponential, logarithmic, or polynomial models.
  - The process involves linearizing the problem or using iterative optimization techniques to minimize the error for complex models.

- **Definition**: Least Squares Regression aims to find an estimation function y_hat(x) that best fits a set of data points (x_i, y_i). The estimation function is typically a linear combination of ‘basis functions’.
- **Estimation Function**: The general form of the estimation function is:

$$\hat{y}(x) = \sum_{i=1}^{n} \alpha_i f_i(x)$$

$$\begin{aligned}
\hat{y}(x_1) &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1), \\
\hat{y}(x_2) &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2), \\
&\cdots \\
\hat{y}(x_m) &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m).
\end{aligned}$$

(m x n matrix)

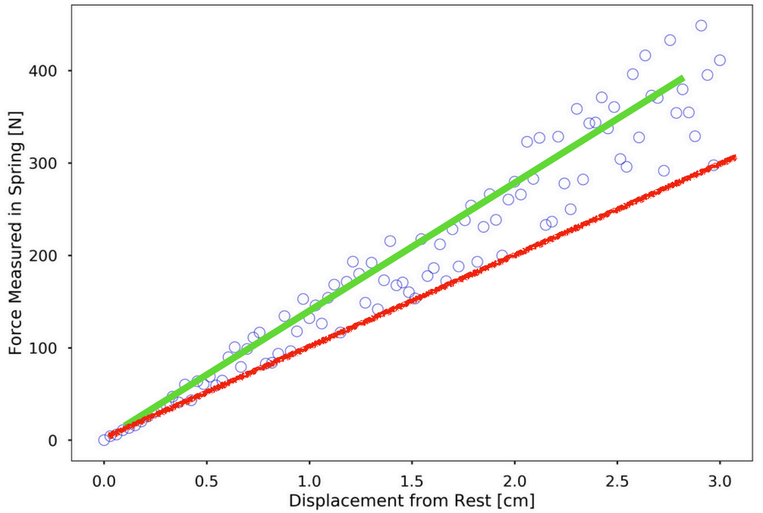
- alpha_i are the parameters to be determined.
- f_i(x) are the basis functions which must be linearly independent (cannot be linear combination of each other).
- Objective: To minimize the total squared error between the estimated and actual data points. The squared error E is defined as a sum over data points:

$$E = \sum_{i=1}^{m} (y_i - \hat{y}(x_i))^2$$

Where:

- y_i is the actual dependent variable.
- y_hat (x_i) is the estimated value from the model at position x_i.

- **Spring System Example**: To better understand the concept, let us consider a force-displacement relationship for a linear spring.



- The **force** F is related to the displacement x by Hooke's Law:

$$F(x) = kx$$

- k is the spring stiffness, which serves as the model parameter.

- Estimation Function: In this case, we know that a good estimation function is simply:

$$\hat{y}(x) = kx$$

- The only basis function here is f(x) = x, and the parameter to be determined is k.

- Goal: Minimize the squared error to find the optimal value of k that fits the force-displacement data.

Minimizing the Total Squared Error

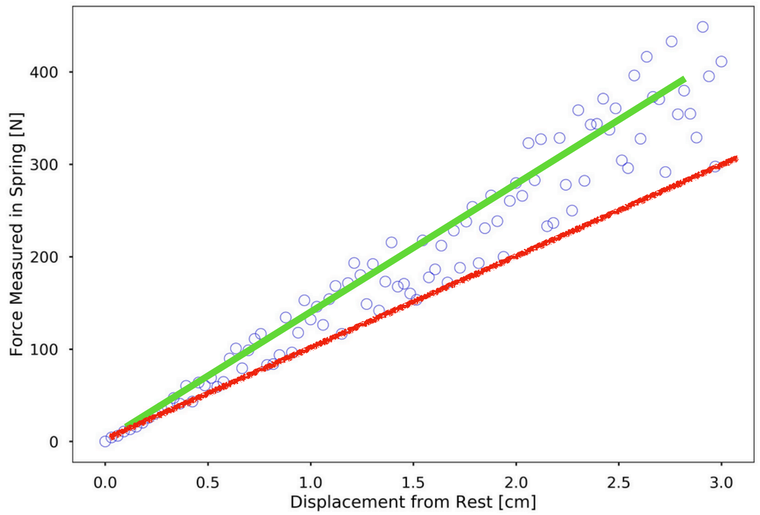
- **Error Definition**: The individual errors (residuals) are defined as:

$$e_i = y_i - \hat{y}(x_i)$$

The objective is to minimize the squared error E, which is essentially the (L2) norm of the residual m-dimensional vector e:

$$E = \|e\|_2^2$$

$$E = \sum_{i=1}^{m} (y_i - \hat{y}(x_i))^2$$



Least Squares Regression - Algebra approach

Objective:

To find the best-fit line for a set of data points by minimizing the total squared error between the observed and estimated values.

Key Concepts:

- **Data Points**: Represented as (x_i, y_i), where x_i is the independent variable and y_i is the dependent variable.
- Goal: Estimate the relationship between x_i and y_i using a linear model Y_hat = A.beta, where A is the matrix of predictor functions, and beta is the vector of alpha coefficients.

Total Squared Error: Minimize the error

$$E = \|Y - \hat{Y}\|^2$$

where Y_hat is the predicted value from the regression model.

$$\hat{Y}_i = \hat{y}(x_i)$$

β be a column vector such that $\beta_i = \alpha_i$

$$\hat{Y} = A\beta$$

Derivation Using Linear Algebra

Vector Notation:

- Let Y be a column vector of observed values and A be a matrix whose columns represent the predictor variables.
- The regression problem becomes solving A.beta = Y by minimizing the total squared error.

Key Equation:

- The total squared error is given by

$$E = \|Y - A\beta\|^2$$

To minimize E, we seek the beta that makes A.beta as close as possible to Y. This can be shown to result in the equation:

$$A^T(Y - A\beta) = 0$$

Recall: $\mathrm{dot}(v, w) = v^T w$

Solution:

- Solving for beta gives the normal equation:

$$\beta = (A^T A)^{-1} A^T Y$$

This is the least squares regression formula, where A^T A is invertible if A has linearly independent columns.

Least Squares Regression - Calculus approach

Recall the Key Concept:

- The goal of least squares regression is to find the set of parameters alpha_j that minimizes the total error E, represented as the sum of squared differences between the predicted values and actual data points.

Equation for Total Error:

$$E = \sum_{i=1}^{m} (y_i - \sum_{j=1}^{n} \alpha_j f_j(x_i))^2$$

Where:

- m = number of data points
- n = number of basis functions
- alpha_j = unknown parameters
- f_j(x_i) = basis functions evaluated at data points x_i
- y_i = actual/target values

Concept of Minimization:

- The error function E is minimized by solving a system of linear equations derived by setting the partial derivatives of E with respect to a specific alpha_k to zero.

Derivation Using Multivariable Calculus

Steps in Derivation:

1. **Partial Derivatives:**

- For each parameter alpha_k, take the partial derivative of the error function E with respect to alpha_k and set it to zero:

$$\frac{\partial E}{\partial \alpha_k} = 2 \sum_{i=1}^{m} \left(y_i - \sum_{j=1}^{n} \alpha_j f_j(x_i) \right) f_k(x_i) = 0$$

2. **Matrix Representation:**

- Consideration all possible k, the system of equations can be expressed in matrix form:

$$A^T A \alpha = A^T Y$$

Where:

- A is the matrix of basis functions evaluated at the data points.
- Y is the column vector of output values.
- alpha is the column vector of parameters to be solved.

3. **Final Solution:**

- Solving the matrix equation leads to the least squares solution:

$$\alpha = (A^T A)^{-1} A^T Y$$

